mCertiKOS Signal Pipeline

Complete Signal Delivery Architecture — User Space ↔ Kernel

PHASE 1 TRIGGER

PHASE 2 TRAP ENTRY

PHASE 3 DISPATCH & DELIVERY

PHASE 4 EXECUTE & RETURN

USER SPACE — Signal Sources (Ring 3)

Shell Process (PID 2)

- `readline() → getchar() → yield()`
- Detects **Ctrl+C** keypress
- Shell receives SIGINT first
- Handler decides kill child

shell_pid = fork(child_pid)

Keyboard Input

- User presses **Ctrl+C**
- PS/2 controller → `mcs_rcv()`
- `cons_getc() → '\x03'` (ETX)
- Shared in consoleintr() buffer

NULL Pointer Access

- `*(int *)0 = 0xDEA1;`
- `mov %ax,%fs:0x0(%edx)`
- EIP=**0x00000000**
- Error: present=0, user=1

Kill() Syscall

- `kill(pid, signum)`
- Explicit signal send
- Used by shell to forward SIGINT(2) / SIGKILL(9) to child

IRQ 1 vector 33

CPU exception #PF (14)

INT 0x30 syscall

—— Ring 3 → Ring 0 —— INT 0x30 (syscall) | INT 14 (page fault) | IRQ 1 (keyboard) ——

Combine saved frame on kernel stack

KERNEL — Trap Layer (kern/trap/)

trap() — Central Entry Point

`kern/trap/trap.c(tf=uint32_ esp,tf=uint32_ eip,...)`

1. CPU saves SS, ESP, EFLAGS, CS, EIP + kernel stack
2. `pushal` saves all general registers
3. Constructs **Trap Frame**
4. Dispatches by `tf->trapno`

Exception (< 32)

Hardware IRQ (trapno ≥ 32)

Trap Frame (tf_t)

Register	Offset	Source
ds, es, fs, gs	+0x00–+0	pushl
EIP, CS	+0x30–2B	segment regs
trapno	+0x28	IDT vector
err	+0x20	CPU/stub
EAX	+0x18	hardcoding trap
EDI	+0x10	scratch registers
EFLAGS	+0x08	Flags
ESP	+0x2C	user stack
SS	+0x30	stack segment

Trap Dispatch Switch

```
switch (tf->trapno) {
    case T_PGFLT (14):   → pgflt_handler()
    case T_SYSCALL (48): → syscall_dispatch()
    case IRQ_KBD (33):   → kbd_proc_data()
    case IRQ_TIMER (32): → thread_yield()
}
```

trapno == 14 (#PF)

trapno == 48 (INT 0x30)

trapno == 33 (IRQ 1 keyboard)

KERNEL — Signal Detection

pgflt_handler()

```
fault_va = rcr2();
if (psti && fault_va == 0x_DEDEA1)
    pending_signals[pid] |= (1 << SIGSEGV);
    return;  // no KERN_PANIC
}
```

- User NULL dereff → **SIGSEGV** pending
- Kernel faults still panic

sys_readline() — Ctrl+C Path

```
c = getchar();  // from consoleIn buffer
if (c == 0x03) {  // ETX = Ctrl+C
    pending_signals[cur_pid] |= (1 << SIGINT);
    return;
}
```

- Shell gets SIGINT on itself
- Shell's handler forwards to child

sys_kill(pid, signum)

```
if (pid <= 0 || pid >= NPR_INFO) return;
if (signum < 1 || signum >= NSIG) return;
pending_signals[pid] |= (1 << signum);
```

- Direct signal injection
- Used for SIGKILL forwarding

SIGSEGV (bit 11)

SIGINT (bit 2)

Other signals (bit 9)

Pending Signals Bitmask

`uint32_t pending_signals[NPR_INFO];`

```
Bits:  31  ...  11  ...  9  ...  2  1  0
       ---  ---  SEGV  ---  ---  INT  ---
```

- One 32-bit word per process
- Each bit = one signal type
- Set asynchronously, checked at trap return

trap() calls before set_pdir_base()

KERNEL — Signal Dispatch Engine (runs BEFORE set_pdir_base)

handle_pending_signals(tf)

Called in trap() before switching to user page table

```
sigmask = pending_signals[cpi];
for (sig = 1; sig < NSIG; sig++) {
    if (sigmask & (1 << sig)) {
        pending_signals[cpi] &= ~(1 << sig);
        sa = &cpu_per[cpi].per[pid].sa[sig];
        if (*sa->sa_handler != NULL)
            deliver_signal(pid, sig, sa, tf);
        else
            default_signal_action(pid, sig, tf);
    }
}
```

For each pending signal

Decision: Handler or Default?

`sigaction_proc(pid)[sig].sa_handler`

- If **NULL** → user registered a handler → `deliver_signal()`
- If **NULL** → no handler → `default_signal_action()`

Registered via `sys_sigaction()` syscall

sa_handler == NULL → default_action

sa_handler != NULL → deliver_signal

default_signal_action()

```
switch (signum) {
    case SIGKILL:
    case SIGSEGV:
    case SIGINT:
    case SIGTERM:
        terminate_process(pid, tf);
        break;
}
```

- SIGKILL: **always** default (uncatchable)
- SIGSEGV/SIGINT: default if no handler

SIGKILL / SIGSEGV / SIGINT terminate

terminate_process(pid, tf)

```
old = mcs_get_state(cpi);
if (old == TSTATE_DEAD) return;
if (old == TSTATE_READY)
    runqueue_remove(READ_IDX, pid);
if (pid == get_cur_mcb())
    PER=set_pcb(0);
```

Queue guard: only remove if READY **Self-check**: only call exit if self

KERNEL — Signal Delivery Engine (User Handler Path)

deliver_signal(pid, sig, sa, tf)

Rewrites trap frame to redirect to user handler:

```
1. user_esp = tf->esp
2. Write trampoline at user_esp:
   B8 12 00 00 00   MOV EAX, 18 (SYS_SIGRETURN)
   CD 30            INT 0x30
   EB FE            JMP $-2 (safety spin)
3. Push: trampoline_addr, signum,
         saved_eax, saved_ptp
4. tf->eip = sa->sa_handler
5. tf->esp = new_esp
```

deliver trampoline & construct stack

tf->eip = handler tf->esp = new stack

User Stack After Rewrite

Offset	Content	Purpose
EBP+30	original ESP	restore pointer
EBP+2C	saved EIP	return address
EBP+28	saved EBP	stack restore
EBP+24	signum	handler argument
EBP+20	trampoline addr	fake return addr
EBP+1c	00 cd 30 00 00 00	INT0x30_00
EBP+1	eb fe 00 00 00	JMP $ebfe, JMP -2

—— Ring 0 → Ring 3 —— IRET (Trap return) —— CPU loads EIP/ESP/EFLAGS from trap frame ——

IRET + handler(signum)

Clone → child (fork() child)

USER SPACE — Execution Outcomes (Ring 3)

User Handler Executes

- CPU jumps to `sa_handler(signum)`
- Handler runs in **user mode**
- Can call `printf()`, `exit()`, `jmp`, ...
- When handler returns → hits trampoline
- Trampoline → `INT 0x30 (sys_sigreturn)`
- **Original context restored transparently**

Process Terminated (Default Action)

- State → **TSTATE_DEAD**
- Removed from ready queue (if READY)
- `thread_yield()` if self-termination
- Scheduler picks next process
- **Never scheduled again**

Scheduler picks next process (child)

Transparent RESTORE: EIP, ESP, EFLAGS

sys_sigreturn()

Trampoline triggers this syscall:

```
saved_ptp = *(user_esp + 4)
saved_eax = *(user_esp + 8)
tf->eip = saved_ptp
tf->esp = saved_esp
```

- Restores original execution context
- Process resumes where it was interrupted
- Transparent to the user program

EIP/ESP restored process continues

Shell Resumes at Prompt

- After handler + signal-aware
- Or after child termination + yield
- Prints `$` prompt
- Accepts next command
- **OS continues running** ✅